Simplicial Complex Multiset Embedders

Algorithms 1 and 2: How They Work and How They Differ

June 12, 2026

Contents

1 The Problem

We are given a *multiset* M of n vectors in $\mathbb{R}^k$. We represent M as a $k \times n$ matrix whose *columns* are the vectors; column order is arbitrary. We want a function

$$f : \mathbb{R}^{n \times k} \longrightarrow \mathbb{R}^N$$

that is:

- **Multiset-invariant**: permuting the columns of M does not change $f(M)$.
- **Injective (embedding)**: distinct multisets map to distinct outputs, i.e. f is an *embedder*, not merely an encoder.
- **Continuous ($\mathbf{C^0}$)**: f is continuous in the entries of X.
- **Positively homogeneous of degree 1 (HomDeg1)**: $f(\lambda X) = \lambda f(X)$ for all $\lambda > 0$.

The two algorithms described here achieve this with output sizes:

	Algorithm 1	**Algorithm 2**
Output size N	$2nk + 1$	$2nk + 1 - k$

Algorithm 2 saves exactly k dimensions. Both are C^0 and HomDeg1 but *not* C^1 (piecewise-linear kinks appear where sorted order changes).

2 Shared Building Blocks

Both algorithms use the same core data structures and sub-steps.

Labels: Eji

Each of the nk matrix entries is identified by a label e_i^j, where $j \in \{0, \ldots, n-1\}$ is the *column* (vector) index and $i \in \{0, \ldots, k-1\}$ is the *row* (coordinate) index. The set of all labels is $E = \{e_i^j : 0 \le j < n,\ 0 \le i < k\}$.

Vertices: Maximal Simplex Vertex (MSV)

An MSV is a subset $V \subseteq E$; equivalently, a $\{0,1\}$-valued $n \times k$ matrix whose 1-entries mark the members of V.

Linear Combinations: Eji_LinComb

An *Eji_LinComb* L is an $n \times k$ matrix of non-negative integer counts obtained by adding MSVs:

$$L = V_{(0)} + V_{(1)} + \cdots + V_{(\ell-1)}, \qquad L[j][i] = \#\big\{s : e_i^j \in V_{(s)}\big\}.$$

A companion integer ℓ (the *index*) records how many MSVs were summed.

Canonical Form

Given an Eji_LinComb L, its *canonical form* $\widetilde{L}$ is obtained by sorting the columns of L's count matrix lexicographically (one column per vector, consistent with the column-vector convention). This mods out the arbitrary labelling of vectors (the j-indices), achieving multiset invariance: two inputs that are column-permutations of each other produce the same canonical Eji_LinCombs.

Hash to Hypercube

Each canonical Eji_LinComb $\widetilde{L}$ is mapped deterministically to a pseudorandom point

$$h\Big(\widetilde{L}\Big) \ \in \ [0,1]^N$$

by hashing its count matrix and index with MD5 and converting the digest bits to floating-point coordinates. Distinct canonical Eji_LinCombs are (with probability 1 over the choice of hash function) mapped to distinct, "generically positioned" points.

Barycentric Subdivision

This sub-step, shared identically by both algorithms, converts a sorted list of gap–MSV pairs into a list of coefficient–Eji_LinComb pairs.

Input: m pairs $(\delta_0, V_0), \ldots, (\delta_{m-1}, V_{m-1})$ with $\delta_0 \ge \delta_1 \ge \cdots \ge \delta_{m-1} \ge 0$.

Output: m pairs (α_i, L_i) where

$$\alpha_i \ = \ (i+1)\big(\delta_i - \delta_{i+1}\big), \qquad \delta_m := 0,$$

$$L_i \ = \ V_0 + V_1 + \cdots + V_i \quad \text{(Eji\_LinComb with index } i{+}1\text{)}.$$

Key properties:

- $\alpha_i \ge 0$ (since gaps are sorted descending).
- $\sum_i \alpha_i = \sum_i \delta_i$ (by Abel summation; the total weight is preserved).
- The L_i are monotone: $L_{i+1} = L_i + V_{i+1} \ge L_i$ entrywise.

Final Output Assembly

Given the barycentric pairs (α_i, L_i), each L_i is canonicalised and hashed to a point $p_i = h\left(\widetilde{L}_i\right) \in [0,1]^{N_{\text{hash}}}$. The main body of the embedding is then

$$\sum_{i=0}^{m-1} \alpha_i \, p_i \;\in\; \mathbb{R}^{N_{\text{hash}}},$$

to which a small number of *anchor values* (global or per-row extrema) are appended or prepended to fix the absolute scale.

Why One Number Can Encode Both Coefficients and Basis Elements

This is the key trick. The balance has the form $\sum_i \alpha_i L_i$ with all $\alpha_i \geq 0$: a point in the *positive cone* generated by the m canonical Eji_LinComb objects $\{L_i\}$. If each L_i is mapped to a generic point $p_i \in \mathbb{R}^{N_{\text{hash}}}$, the positive cone of $\{p_0, \ldots, p_{m-1}\}$ has dimension m. Two m-dimensional cones in $\mathbb{R}^{2m+1}$ are *generically disjoint* by dimension counting: $m + m = 2m < 2m + 1$, so their intersection has negative expected dimension and is (with probability 1) empty. Therefore:

- A point $p = \sum_i \alpha_i p_i$ in the interior of the cone uniquely identifies *which* $\{p_i\}$ were used (i.e. which canonical $\{L_i\}$, i.e. which basis elements).
- Once the cone is identified, the α_i are the unique non-negative coefficients expressing p in that cone.

This is why $N_{\text{hash}} = 2m + 1$: it is the minimum ambient dimension for m positive cones to be generically disjoint.

Connection to continuity and the simplicial complex. When $\alpha_i = 0$ for some i, the point p lies on a *face* of the cone (a sub-cone with one fewer generator). Two different cones share that face if they agree on all basis elements except the one with zero coefficient. This is exactly the geometry of a *simplicial complex*: higher-dimensional simplices glued along lower-dimensional faces. And these shared faces are precisely the continuity points discussed earlier: the transition in sorted order (which changes the basis-element identity) happens at the same moment as the coefficient goes to zero. The two events cancel each other, keeping the output continuous, and the shared face is the geometric expression of that cancellation.

3 Algorithm 1: Global Sort

Source file: `C0HomDeg1_simplicialComplex_embedder_1_for_array_of_reals_as_multiset.py`
Output size: $N = 2nk + 1$

Steps

1. **Global sort.** Treat all nk matrix entries as labelled scalars and sort them in *descending* order:
$$v_0 \;\geq\; v_1 \;\geq\; \cdots \;\geq\; v_{nk-1}, \quad v_r = X[j_r][i_r], \quad e_r = e_{i_r}^{j_r}.$$

2. **Gaps and prefix MSVs.** Compute the $nk - 1$ consecutive differences $\delta_r = v_r - v_{r+1}$ for $r = 0, \ldots, nk - 2$. Associate each gap r with the *prefix MSV* $V_r = \{e_0, e_1, \ldots, e_r\}$ (the set of labels of the top $r{+}1$ values).

3. **Re-sort by gap size.** Sort the $nk - 1$ pairs (δ_r, V_r) in descending order of δ_r.

4. **Barycentric subdivision** (Section 2) with $m = nk - 1$.

5. **Canonicalise and hash.** For each (α_i, L_i), compute $\widetilde{L}_i$ and $p_i = h\left(\widetilde{L}_i\right)$ with $N_{\text{hash}} = 2(nk-1)+1 = 2nk-1$.

6. **Assemble output.**

$$f_1(X) = \Big[\underbrace{\textstyle\sum_i \alpha_i\, p_i}_{2nk-1 \text{ reals}}, \quad \underbrace{v_0}_{\text{global max}}, \quad \underbrace{v_{nk-1}}_{\text{global min}} \Big] \in \mathbb{R}^{2nk+1}.$$

Note on the maximum. Given the global minimum and all $nk-1$ gaps, one can reconstruct all nk values; storing the global maximum is technically redundant. It is included for implementation simplicity (a TODO comment in the code acknowledges this for degenerate cases).

Basis-change and continuity picture. Algorithm 1 performs the same style of basis change as Algorithm 2 (Section 4, "What the Steps Are Actually Doing"), but with one global sort instead of k per-row sorts. The global minimum is the coefficient of the single fully-symmetric basis element $\sum_{i,j} e_i^j$; the prefix MSVs $V_r = \{e_0, \ldots, e_r\}$ are the sets of the top-$(r{+}1)$ entries across *all* rows and vectors combined. Continuity follows by the same argument: any gap that goes to zero signals an equality between two globally adjacent entries, and at that moment the coefficient of the transitioning basis element is zero.

Worked Example: $n = 4$, $k = 2$

Input multiset (four 2-vectors shown as column vectors):

$$M = \left\{ \begin{pmatrix}4\\2\end{pmatrix}, \begin{pmatrix}-3\\5\end{pmatrix}, \begin{pmatrix}8\\9\end{pmatrix}, \begin{pmatrix}2\\7\end{pmatrix} \right\}$$

Step 1–2: Global sort and gaps.

Rank r	Value v_r	Label e_r	Gap $\delta_r = v_r - v_{r+1}$
0	9	e_1^2	1
1	8	e_0^2	1
2	7	e_1^3	2
3	5	e_1^1	1
4	4	e_0^0	2
5	2	e_1^0	0
6	2	e_0^3	5
7	−3	e_0^1	—

Global max = 9, global min = −3. The 7 prefix MSVs are $V_r = \{e_0, \ldots, e_r\}$; for example $V_2 = \{e_1^2, e_0^2, e_1^3\}$.

Step 3: Re-sort by gap size. After sorting the 7 pairs by δ descending: gaps of size 5, 2, 2, 1, 1, 1, 0.

Steps 4–5: Barycentric subdivision. Non-zero α_i values (zero terms contribute nothing to the sum):

$$\alpha_0 = 1 \cdot (5-2) = 3, \quad \alpha_2 = 3 \cdot (2-1) = 3, \quad \alpha_5 = 6 \cdot (1-0) = 6.$$

The remaining $\alpha_i = 0$.

Step 6: Output. $N_{\text{hash}} = 2(8-1)+1 = 15$.

$$f_1(X) = 3\,p_0 + 3\,p_2 + 6\,p_5 + [\,9,\ -3\,] \ \in\ \mathbb{R}^{17}.$$

where each $p_i \in [0,1]^{15}$ is the hash of the corresponding canonical Eji_LinComb. The full numerical output is:

[7.07, 6.91, 2.78, 3.50, 1.01, 5.51, 4.07, 4.90, 10.02, 9.51, 5.13, 2.97, 8.04, 5.34, 7.41, 9, −3].

4 Algorithm 2: Per-Component Sort

Source file: `C0HomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py`
Output size: $N = 2nk + 1 - k$

Steps

1. **Per-row sort.** For each coordinate $i = 0, \ldots, k-1$, sort the n values in row i in *descending* order:
$$X[\sigma_i(0)][i] \ \geq\ X[\sigma_i(1)][i] \ \geq\ \cdots \ \geq\ X[\sigma_i(n-1)][i].$$

2. **Per-row gaps and prefix MSVs.** For each row i, compute the $n-1$ consecutive differences within that row:
$$\delta_r^{(i)} = X[\sigma_i(r)][i] - X[\sigma_i(r+1)][i], \quad r = 0, \ldots, n-2.$$
Associate gap r in row i with the prefix MSV $V_r^{(i)} = \{e_i^{\sigma_i(0)}, \ldots, e_i^{\sigma_i(r)}\}$ (all entries *within row i*; no cross-row mixing). Record the k row minima: $m_i = X[\sigma_i(n-1)][i]$.

3. **Flatten and re-sort.** Collect all $k(n-1) = nk - k$ pairs $(\delta_r^{(i)},\ V_r^{(i)})$ into one list and sort descending by gap size.

4. **Barycentric subdivision** (Section 2) with $m = nk - k$.

5. **Canonicalise and hash.** For each (α_i, L_i), compute $\widetilde{L}_i$ and $p_i = h\big(\widetilde{L}_i\big)$ with $N_{\text{hash}} = 2k(n-1)+1 = 2nk - 2k + 1$.

6. **Assemble output.**
$$f_2(X) \ =\ \Big[\ \underbrace{m_0, \ldots, m_{k-1}}_{k \text{ row minima}},\ \underbrace{\textstyle\sum_i \alpha_i\, p_i}_{2nk-2k+1 \text{ reals}}\ \Big] \ \in\ \mathbb{R}^{2nk+1-k}.$$

Why no row maxima? The maximum of each row (coordinate) is implicitly encoded: it equals the row minimum plus the sum of that row's gaps (which are embedded in the hash-weighted sum). Only the row minima are needed as anchors to fix the absolute scale.

What the Steps Are Actually Doing

Steps 1–2 are a change of basis, not just bookkeeping. Write the multiset as $M = \sum_{i,j} x_i^j\, e_i^j$. For each row i, the row minimum m_i is the coefficient of the fully symmetric basis element $\sum_j e_i^j$, which is invariant under all permutations of the vectors. Factoring it out:

$$M \ =\ \underbrace{\sum_i m_i \Big(\sum_j e_i^j\Big)}_{\text{symmetric part, stored as anchors}} \ +\ \underbrace{\sum_{i,r} \delta_r^{(i)}\ V_r^{(i)}}_{\text{balance, all coefficients} \geq 0}$$

where the second equality is the Abel (summation-by-parts) identity applied within each row. The row minima are recorded directly as anchor values because they multiply *fully symmetric* basis elements and are therefore already permutation-invariant.

The specific structure of $V_r^{(i)}$ is what matters. $V_r^{(i)} = \{e_i^{\sigma_i(0)}, \ldots, e_i^{\sigma_i(r)}\}$ is not an arbitrary subset of row i: it is the set of the $r{+}1$ vectors that have the *largest* values in row i, determined by the sorted order σ_i. The identity of those vectors (which j's appear) is the non-symmetric information that cannot yet be discarded. It is recorded inside the Eji_LinCombs L_r and ultimately protected by the hash step.

Continuity across sorted-order changes. The algorithm is C^0 even though sorted orders change discontinuously. The reason: when two values within the same row are equal ($X[\sigma_i(r)][i] = X[\sigma_i(r+1)][i]$), the gap at that position is exactly 0, so the coefficient of the corresponding basis element $V_r^{(i)}$ is exactly 0. At the very moment the sorted order (and hence the identity of $V_r^{(i)}$) would switch, we need zero of whichever basis element we are assigning it to. The zero coefficient absorbs the discontinuity in the basis-element identity, keeping the output continuous. The same argument applies to the global re-sort in Step 3: when two gaps from different rows are equal, the barycentric coefficient at that rank is 0, so it does not matter which order those two gaps are listed in.

Worked Example: $n = 4$, $k = 3$

Input multiset (four 3-vectors shown as column vectors):

$$M = \left\{ \begin{pmatrix} 4 \\ 2 \\ 3 \end{pmatrix}, \begin{pmatrix} -3 \\ 5 \\ 1 \end{pmatrix}, \begin{pmatrix} 8 \\ 9 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ 7 \\ 2 \end{pmatrix} \right\}$$

Row-0 (coordinate 0) minima: $m_0 = -3$, row-1: $m_1 = 2$, row-2: $m_2 = 1$.

Steps 1–2: Per-row sorts and gaps.

Row i	Sorted desc. (value, label)	Gaps δ	Prefix MSVs (for each gap)
0	$8(e_0^2)$, $4(e_0^0)$, $2(e_0^3)$, $-3(e_0^1)$	4, 2, 5	$\{e_0^2\}$; $\{e_0^2, e_0^0\}$; $\{e_0^2, e_0^0, e_0^3\}$
1	$9(e_1^2)$, $7(e_1^3)$, $5(e_1^1)$, $2(e_1^0)$	2, 2, 3	$\{e_1^2\}$; $\{e_1^2, e_1^3\}$; $\{e_1^2, e_1^3, e_1^1\}$
2	$3(e_2^0)$, $2(e_2^2)$, $2(e_2^3)$, $1(e_2^1)$	1, 0, 1	$\{e_2^0\}$; $\{e_2^0, e_2^2\}$; $\{e_2^0, e_2^2, e_2^3\}$

Total of $3 \times 3 = 9 = nk - k$ pairs (3 per row).

Step 3: Flatten and re-sort by gap size.

Rank r	Gap δ_r	MSV V_r	Origin
0	5	$\{e_0^2, e_0^0, e_0^3\}$	row 0, gap 2
1	4	$\{e_0^2\}$	row 0, gap 0
2	3	$\{e_1^2, e_1^3, e_1^1\}$	row 1, gap 2
3	2	$\{e_0^2, e_0^0\}$	row 0, gap 1
4	2	$\{e_1^2\}$	row 1, gap 0
5	2	$\{e_1^2, e_1^3\}$	row 1, gap 1
6	1	$\{e_2^0\}$	row 2, gap 0
7	1	$\{e_2^0, e_2^2, e_2^3\}$	row 2, gap 2
8	0	$\{e_2^0, e_2^2\}$	row 2, gap 1

Step 4: Barycentric subdivision. Gap sequence: $5, 4, 3, 2, 2, 2, 1, 1, 0$.

$$\alpha_r = (r{+}1)(\delta_r - \delta_{r+1}): \quad 1,\ 2,\ 3,\ 0,\ 0,\ 6,\ 0,\ 8,\ 0.$$

Non-zero terms only:

$\alpha_0 = 1$ (uses L_0), $\alpha_1 = 2$ (uses L_1), $\alpha_2 = 3$ (uses L_2), $\alpha_5 = 6$ (uses L_5), $\alpha_7 = 8$ (uses L_7).

The Eji_LinComb count matrices $L_r = V_0 + \cdots + V_r$ for the non-zero terms are (showing index / count matrix, before canonicalisation):

Term	Count matrix (rows = coordinates i=0, 1, 2; cols = vectors j=0, 1, 2, 3)
L_0 (idx 1)	$\begin{pmatrix} 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$
L_1 (idx 2)	$\begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$
L_2 (idx 3)	$\begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$
L_5 (idx 6)	$\begin{pmatrix} 2 & 0 & 3 & 1 \\ 0 & 1 & 3 & 2 \\ 0 & 0 & 0 & 0 \end{pmatrix}$
L_7 (idx 8)	$\begin{pmatrix} 2 & 0 & 3 & 1 \\ 0 & 1 & 3 & 2 \\ 2 & 0 & 1 & 1 \end{pmatrix}$

Step 6: Output. $N_{\text{hash}} = 2 \times 3 \times 3 + 1 = 19$.

$$f_2(X) = \big[\ \underbrace{-3,\ 2,\ 1}_{\text{3 row min.}},\ 1\,p_0 + 2\,p_1 + 3\,p_2 + 6\,p_5 + 8\,p_7\ \big] \ \in\ \mathbb{R}^{22},$$

where each $p_r \in [0, 1]^{19}$.

5 Comparison

Side-by-Side Summary

	Algorithm 1	Algorithm 2
Sort strategy	One global sort of all nk entries	k independent row sorts
Number of gaps	$nk - 1$	$k(n-1) = nk - k$
MSV structure	Prefix sets of the global order; MSVs can mix entries from any row	Prefix sets within each row; all entries in a given MSV share the same row (i-index)
Hash embedding dim.	$2(nk-1)+1 = 2nk-1$	$2k(n-1)+1 = 2nk-2k+1$
Anchor values	Global max and global min (2 values)	Per-row minima (k values)
Total output size	$(2nk-1)+2 = 2nk+1$	$(2nk-2k+1)+k = 2nk+1-k$
Saving vs. Alg 1	—	k dimensions

Why Algorithm 2 is Smaller

Algorithm 2 saves k output dimensions. The saving arises from two interacting changes.

1. **Fewer gaps to embed ($k-1$ fewer).** Sorting per-row gives $k(n-1)$ gaps; sorting globally gives $nk-1$ gaps. The difference is $k-1$. Each gap costs 2 hash dimensions (since $N_{\text{hash}} = 2m+1$), so the hash embedding shrinks by $2(k-1)$.
2. **More anchor values ($k-2$ more for $k > 2$).** Algorithm 1 stores 2 anchors (global max and min). Algorithm 2 stores k anchors (per-row minima), which is $k-2$ more for $k > 2$. This partially offsets the hash savings.

Net saving: $2(k-1) - (k-2) = k$. This holds for all $k \geq 1$.

The *reason* Algorithm 2 needs $k-1$ fewer gaps is conceptual: in the global sort (Algorithm 1), the very last (smallest) entry has no gap below it within the chain; that "missing" gap is represented by the global minimum anchor. In Algorithm 2, the same thing happens *independently in each row*: the smallest entry of each row has no gap below it in that row, and each such "missing" gap is represented by one of the k row-minimum anchors. Algorithm 2 exploits the per-row structure to reduce the number of "missing" gaps from one (global) to k (one per row), which at first seems *worse* — but the per-row sorting also prevents $k-1$ cross-row comparisons from appearing in the gap list, yielding a net reduction of $k-1$ gaps and ultimately a saving of k output dimensions.

Output Size and the Whitney Embedding Theorem

Both algorithms produce output of size $\approx 2nk$, which is not obviously wasteful. The input space — multisets of n vectors in $\mathbb{R}^k$ — is an nk-dimensional manifold (the symmetric product $SP^n(\mathbb{R}^k)$). The **Whitney Embedding Theorem** states that any smooth d-dimensional manifold can be smoothly embedded into $\mathbb{R}^{2d}$, and that $2d$ is in general tight. Output size $\approx 2nk$ is therefore right at the Whitney bound: anything below $2nk$ would be provably impossible for a smooth embedding of a generic nk-dimensional input space.

The factor of ≈ 2 is not overhead to be engineered away; it is intrinsic to what the algorithms are achieving. Simultaneously encoding *both* the positive coefficients α_i *and* the identities of the basis elements L_i inside a single point in $\mathbb{R}^{2m+1}$ (Section 2, "Why One Number Can Encode Both…") requires exactly $2m + 1$ dimensions. This is also the mechanism that delivers C^0 continuity. The factor of two in the output size is the price of those properties — and by Whitney it is the minimum such price for a smooth embedding.

Whether $2nk$ (or $2nk + 1$, or $2nk + 1 - k$) is optimal for the specific space $SP^n(\mathbb{R}^k)$, or whether some improvement is achievable using the known structure of that space, remains an open question.